

Universidad ORT Uruguay
Facultad de Ingeniería

Diseño de Aplicaciones II
Obligatorio 1

Descripción del Diseño

Santiago Duró - 256325 - M6C

Lucas Facello - 298679 - M6C

Federico Katz - 257073 - M6B

Docentes M6C: Nicolás Fierro, Alexander Wieler

Docentes M6B: Daniel Acevedo, Federico González

Repositorio:

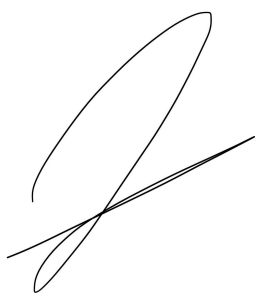
https://github.com/IngSoft-DA2/257073_256325_298679

8 de octubre, 2024

DECLARACIÓN DE AUTORÍA

Nosotros, Santiago, Federico, y Lucas, declaramos que el trabajo que se presenta en esa obra es de nuestra propia mano. Podemos asegurar que:

- La obra fue producida en su totalidad mientras realizábamos el curso de Diseño de Aplicaciones II;
- Cuando hemos consultado el trabajo publicado por otros, lo hemos atribuido con claridad;
- Cuando hemos citado obras de otros, hemos indicado las fuentes. Con excepción de estas citas, la obra es enteramente nuestra;
- En la obra, hemos acusado recibo de las ayudas recibidas;
- Cuando la obra se basa en trabajo realizado conjuntamente con otros, hemos explicado claramente qué fue contribuido por otros, y qué fue contribuido por nosotros;
- Ninguna parte de este trabajo ha sido publicada previamente a su entrega, excepto donde se han realizado las aclaraciones correspondientes.



Santiago Duró

8 de octubre 2024



Lucas Facello

8 de octubre 2024



Federico Katz

8 de octubre 2024

Índice

Descripción del trabajo.....	4
Descripción general.....	4
Descripción del sistema.....	4
Errores Conocidos / Dudas Técnicas.....	5
Diagramas.....	7
Diagrama de Paquetes.....	7
Diagramas de LogicaNegocio.....	8
Diagrama de Persistencia.....	10
Diagramas de WebApi.....	12
Modelo de Tablas.....	13
Diagramas de Interacción.....	14
Justificación del Diseño.....	16
Patrón de Diseño Adapter.....	17
Diagrama de Componentes.....	19

Descripción del trabajo

Descripción general

Este proyecto fue realizado utilizando los entornos de desarrollo Rider y Visual Studio, como también GitHub para el versionado del mismo. El lenguaje utilizado fue .NetCore versión 8.0. El manejo de la base de datos fue a través de Microsoft SQL Server Management Studio 2022.

Descripción del sistema

Este trabajo consiste en un sistema que permita centralizar y gestionar diferentes dispositivos inteligentes en un hogar, creando un entorno integrado y automatizado. Dentro del sistema, hay implementados tres diferentes roles que pueden tener los usuarios: administrador, dueño de empresa y dueño de hogar. Cada uno de ellos consta de una lista de permisos diferente, la cual los habilita a realizar distintas acciones dentro del programa.

Al comenzar, el administrador creado por defecto debe autenticarse para luego poder realizar alguna de las funcionalidades que este tiene habilitadas. Este es quien tiene más funcionalidades habilitadas, ya que es quien gestiona la aplicación. Un administrador tiene la capacidad de crear otros administradores, como también eliminarlos. Puede listar todos los usuarios y empresas registradas en el sistema. Por último, el administrador es quien puede registrar a un usuario con rol dueño de empresa.

El rol de dueño de empresa es responsable de registrar empresas en el sistema. Cada dueño de empresa puede tener una única empresa asociada. Además, es quien gestiona el alta de los dispositivos vinculados a su empresa.

Los usuarios no autenticados pueden registrarse en el sistema con el rol de dueño de hogar, lo que les permite crear y gestionar múltiples hogares inteligentes. Una vez que un dueño de hogar registra un hogar, puede agregar miembros con distintos niveles de permisos. Estos permisos incluyen la capacidad de asociar dispositivos, listar los dispositivos conectados y gestionar las notificaciones asociadas al hogar.

Sin embargo, hay funciones que pueden ser realizadas por todos los usuarios autenticados, sin importar su rol predefinido. Una de ellas sería la posibilidad de listar todos los dispositivos registrados en el sistema. También pueden obtener todos los tipos de dispositivos que soporta el sistema, para esta entrega solo se nos indicó que los soportados son sensores y cámaras inteligentes.

El diseño de nuestro proyecto se basó en una estructura vertical. Es decir, realizamos las diferentes partes del sistema en función de cada recurso necesario para cada uno de los

requerimientos indicados. En lugar de implementar toda la lógica de negocio de una vez y luego enfocarnos en la persistencia, adoptamos un enfoque incremental. A medida que introducimos nuevos recursos o funcionalidades, construimos su respectiva lógica de negocio y su persistencia de forma simultánea.

Este enfoque de desarrollo incremental y vertical nos permitió ir cumpliendo con los requerimientos de manera más eficiente, enfocándonos en un recurso a la vez y asegurando que estuviera completamente integrado con el sistema antes de avanzar.

Para esta primera entrega, se nos pidió solo el backend de la aplicación. Por lo tanto para poder probar las diferentes funcionalidades de nuestra API REST fue necesario utilizar Postman para corroborar el correcto funcionamiento de lo implementado.

Esta parte del sistema la implementamos luego de desarrollar la parte de lógica de negocio y de persistencia del requerimiento a desarrollar, asegurándonos una buena separación de responsabilidades como también una estabilidad acorde.

Para diseñar el sistema utilizamos la metodología TDD (Test Driven Development). Sin embargo, en algunos casos se desarrollaron algunas validaciones en primer lugar y luego creamos los tests correspondientes que cubren la totalidad del código. Esto fue con el objetivo de acelerar el desarrollo y no bloquear el avance entre los integrantes del equipo. Cabe destacar que por cada paquete existe el correspondiente paquete de tests.

A lo largo del desarrollo del programa buscamos constantemente seguir las buenas prácticas de clean code, aplicando diferentes principios SOLID. La forma de realizar los merge a develop nos ayudó a seguir constantemente con un código limpio y ordenado, ya que sino no era posible avanzar. Con respecto a los principios SOLID, podemos destacar el uso del Single Responsibility Principle (SRP). Intentamos dejar cada clase e interfaz con una única responsabilidad clara, con métodos cortos y entendibles. En el sistema también se puede apreciar el Open Closed Principle (OCP). Implementamos los roles y permisos de los usuarios para que se puedan agregar nuevos sin tener que hacer grandes cambios en el código. Otro principio tenido en cuenta es el Interface Segregation Principle (ISP), ya que el programa fue diseñado mediante el uso de interfaces específicas. Por último, destacar el uso del Dependency Inversion Principle (DIP) ya que las clases de alto nivel no dependen de las de bajo nivel.

Errores Conocidos / Dudas Técnicas

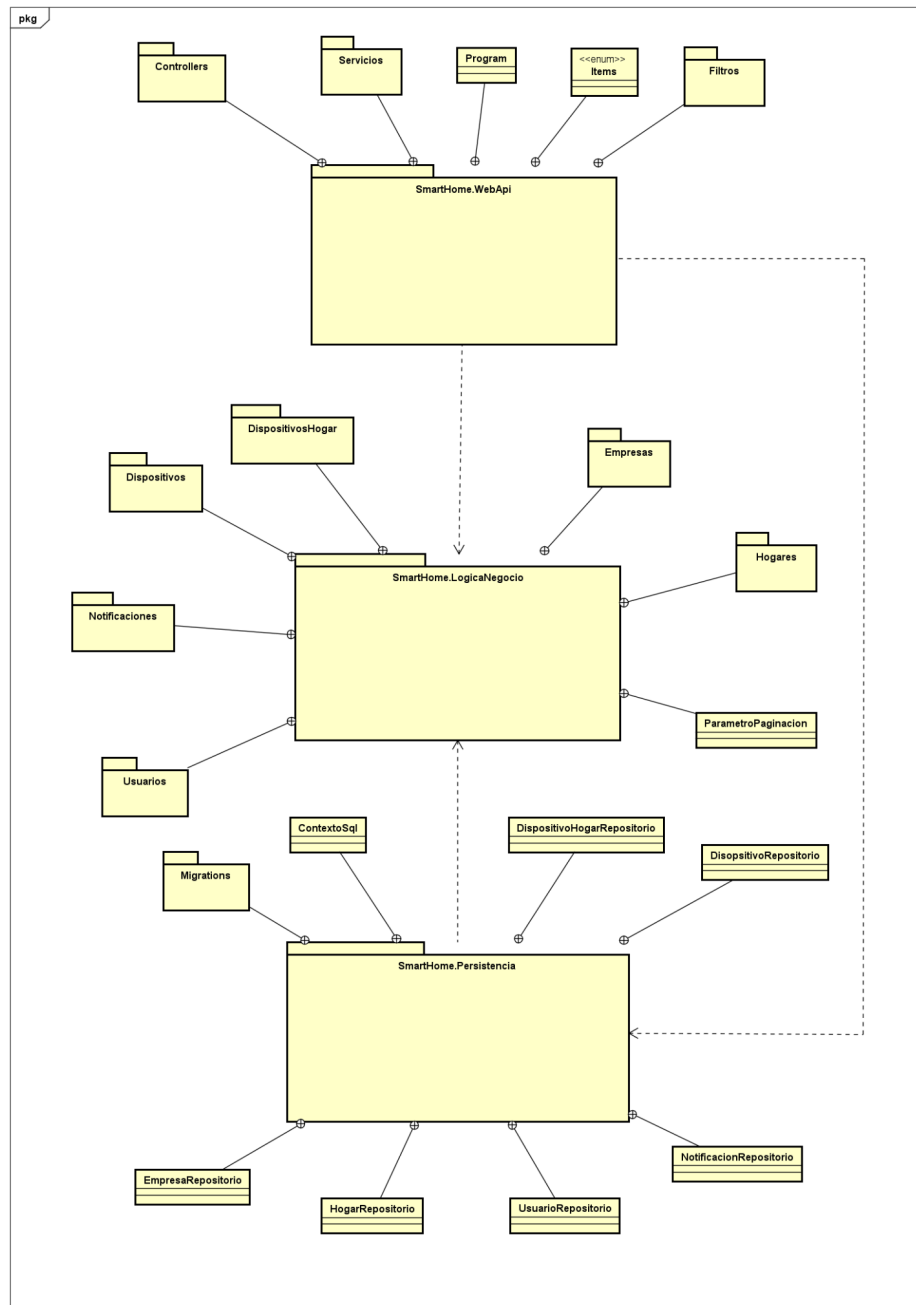
- Una validación del filtrado y la paginación (lógica de negocio) quedó implementada dentro de la capa de persistencia. Esto rompe la separación de responsabilidades entre el acceso a datos y la lógica de negocio, haciendo que se dificulte la mantenibilidad, reduce la reutilización del código, y también complica la realización de pruebas unitarias.
- Algunos de los últimos commits abarcaron más cambios de los necesarios. Al implementar ciertos requerimientos, a menudo surgía la necesidad de modificar otros

recursos para probar su funcionamiento, lo que resulta en cambios en una rama que no estaba relacionada específicamente con ese requerimiento. Esto provocó que se realizarán modificaciones en muchos archivos sin relación directa con la rama en la que se estaba trabajando, lo que terminó afectando el correcto uso de GitFlow.

- En algunos controladores de la Web API, se ha incluido demasiada lógica que debería estar delegada a servicios o la capa de Lógica de Negocio. Esto genera controladores cargados, complicando su mantenibilidad.
- Actualmente, los permisos se gestionan como un Enum en el código, donde cada rol tiene una lista de permisos necesarios. Si bien esto es funcional, limita la extensibilidad a futuro, ya que cualquier cambio o adición de nuevos permisos implica modificar el código y desplegar nuevamente.

Diagramas

Diagrama de Paquetes



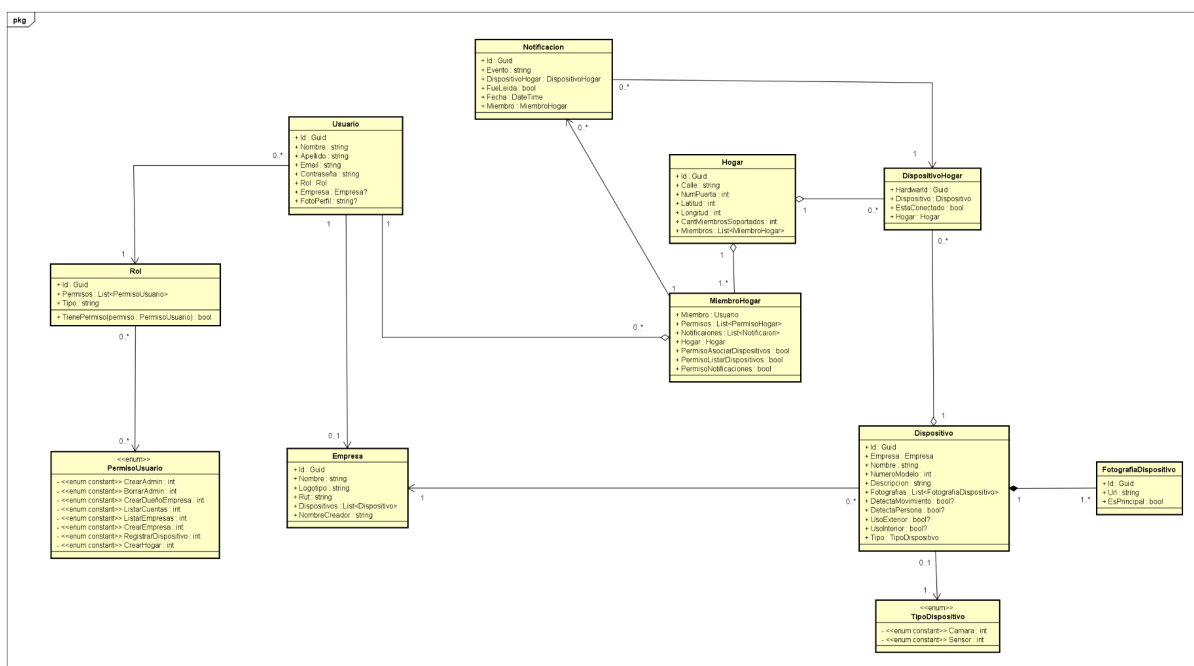
Paquete WebApi: Actúa como la capa de presentación, recibiendo solicitudes HTTP, procesándolas y devolviendo respuestas a los clientes. Los Controllers gestionan las solicitudes del cliente y se comunican con la lógica de negocio. Los Servicios se utilizan para implementar lógica común que pueda ser compartida entre varios controladores. Los Filtros gestionan la interceptación de solicitudes o respuestas para aplicar validaciones o autenticación.

Paquete LogicaNegocio: Implementa toda la lógica y reglas de negocio del sistema, aplicando las reglas específicas para cada funcionalidad. Las clases dentro de este paquete se encargan de gestionar dispositivos, hogares, usuarios, notificaciones y empresas.

Dentro de este paquete optamos por incluir tanto los servicios como el dominio, aumentando la cohesión y disminuyendo el acoplamiento. Utilizamos una arquitectura vertical para fomentar el reuso de las features.

Paquete Persistencia: Gestiona el acceso a la base de datos y la persistencia de los objetos. Utiliza repositorios para abstraer el acceso a la base de datos, garantizando que la lógica de negocio no dependa de implementaciones específicas de almacenamiento.

Diagramas de LogicaNegocio



Dentro del dominio de nuestro sistema podemos encontrar todas las clases utilizadas. En este punto nos parece relevante destacar ciertos aspectos que tuvimos en cuenta a la hora de diseñar nuestro dominio.

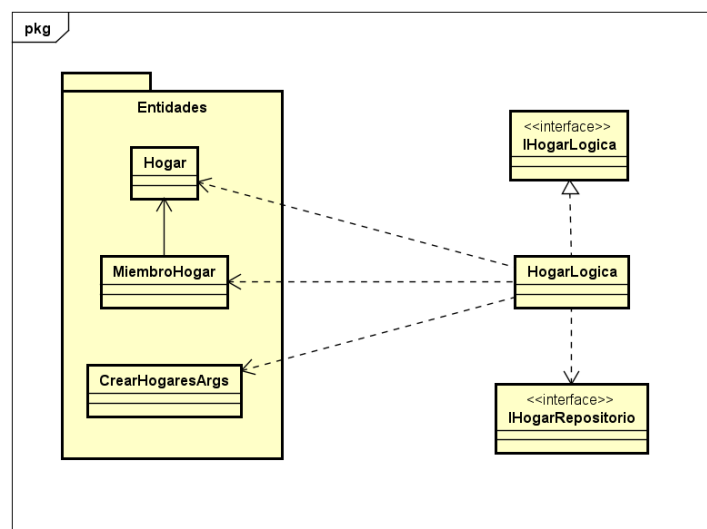
- Luego de discutir, decidimos hacer una clase sola Usuario, pero que uno de los atributos de la misma sea otra clase Rol. En esta clase tenemos un Id hardcodeado por defecto para cada uno de los roles pedidos para esta entrega. Como mostramos debajo, decidimos cargar en la seed data roles predefinidos con los Ids mencionados anteriormente para asegurarnos la consistencia de la aplicación cada vez que se levanta la base de datos. Por último, decidimos implementar una Lista de Permisos, donde se especifica cuales son las acciones que puede realizar cada uno de los roles del sistema. Esta última clase la definimos como un enum, donde tenemos todas las funcionalidades que pueden llevar a cabo los usuarios.

RolesPredefinidos	
+	<u>ID DUEÑO HOGAR : Guid</u>
+	<u>ID DUEÑO EMPRESA : Guid</u>
+	<u>ID ADMIN : Guid</u>
+	<u>Admin : Rol</u>
+	<u>DueñoEmpresa : Rol</u>
+	<u>DueñoHogar : Rol</u>

- La forma en la que manejamos los dispositivos asociados a un hogar. Decidimos crear una clase intermediaria entre Dispositivo y Hogar, donde guardamos el dispositivo que se asocia y el hogar al que está asociado.
- También utilizamos el mismo razonamiento para manejar los miembros asociados al hogar, con una tabla intermedia entre Usuario y Hogar.
- En un principio, pensamos en establecer la relación entre MiembroHogar y Notificacion como N a N, donde cada vez que se generaba una notificación, ésta se guardaba una lista con los miembros que podían leerla. Durante la implementación, tuvimos que cambiar esta relación ya que las notificaciones tienen como atributo si fueron leídas, y si contienen una lista de miembros, no podríamos representar que algunos miembros la leyeron y otros no. Debido a esto, ahora se genera una notificación individual para cada miembro del hogar.

Para cada feature dentro de la lógica del negocio, utilizamos una estructura similar: una carpeta con las clases del dominio que serán implementadas por dicha feature, y los archivos con la lógica, la interfaz de la lógica y la interfaz del repositorio.

Si lo llevamos al caso concreto de la feature Hogares (para el resto de features, se utiliza el mismo razonamiento), nos encontraríamos con el siguiente diagrama:



En cuanto a los métodos que debe implementar cada lógica concreta, estos son definidos por sus respectivas interfaces:

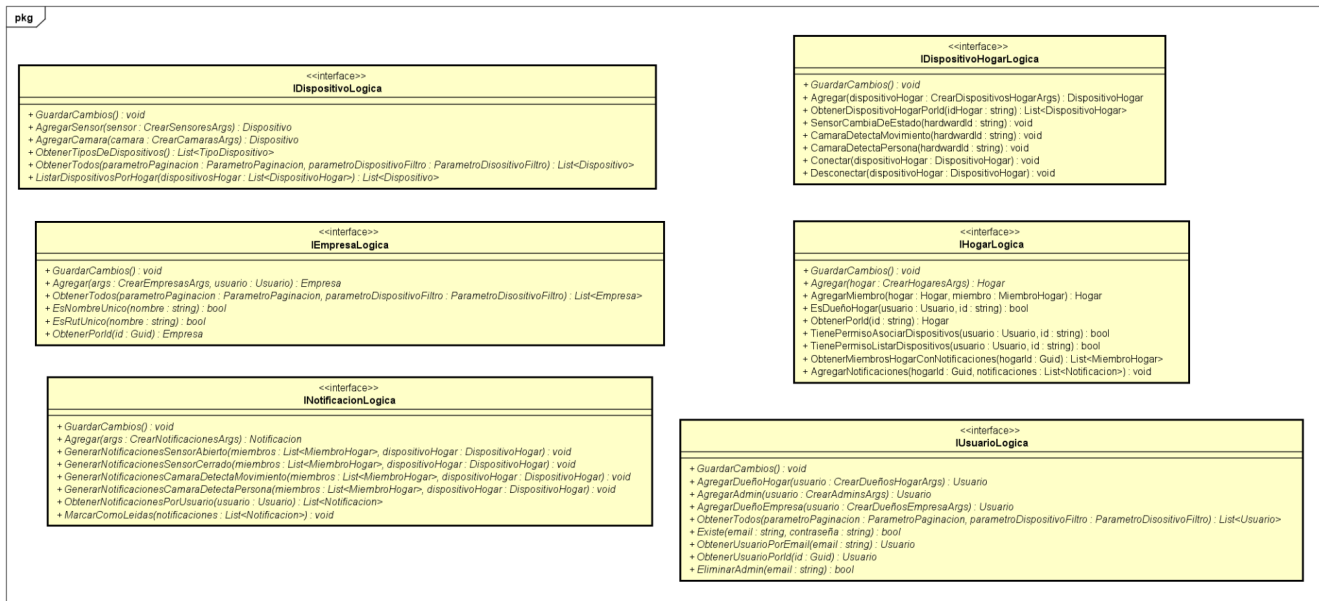
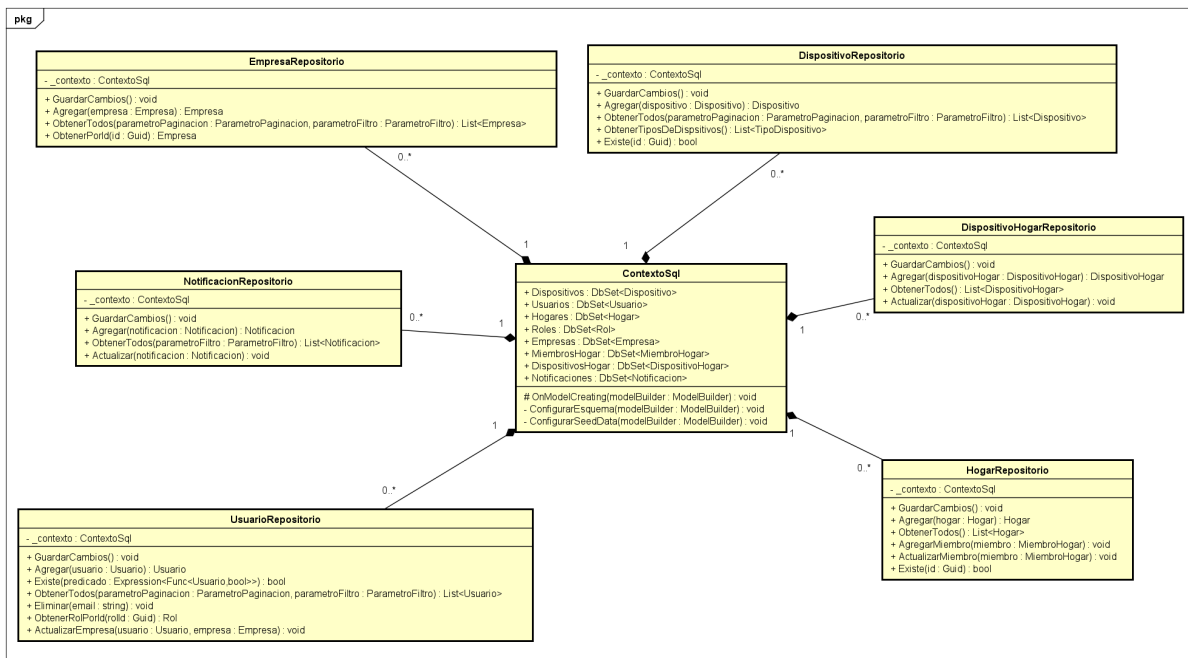


Diagrama de Persistencia

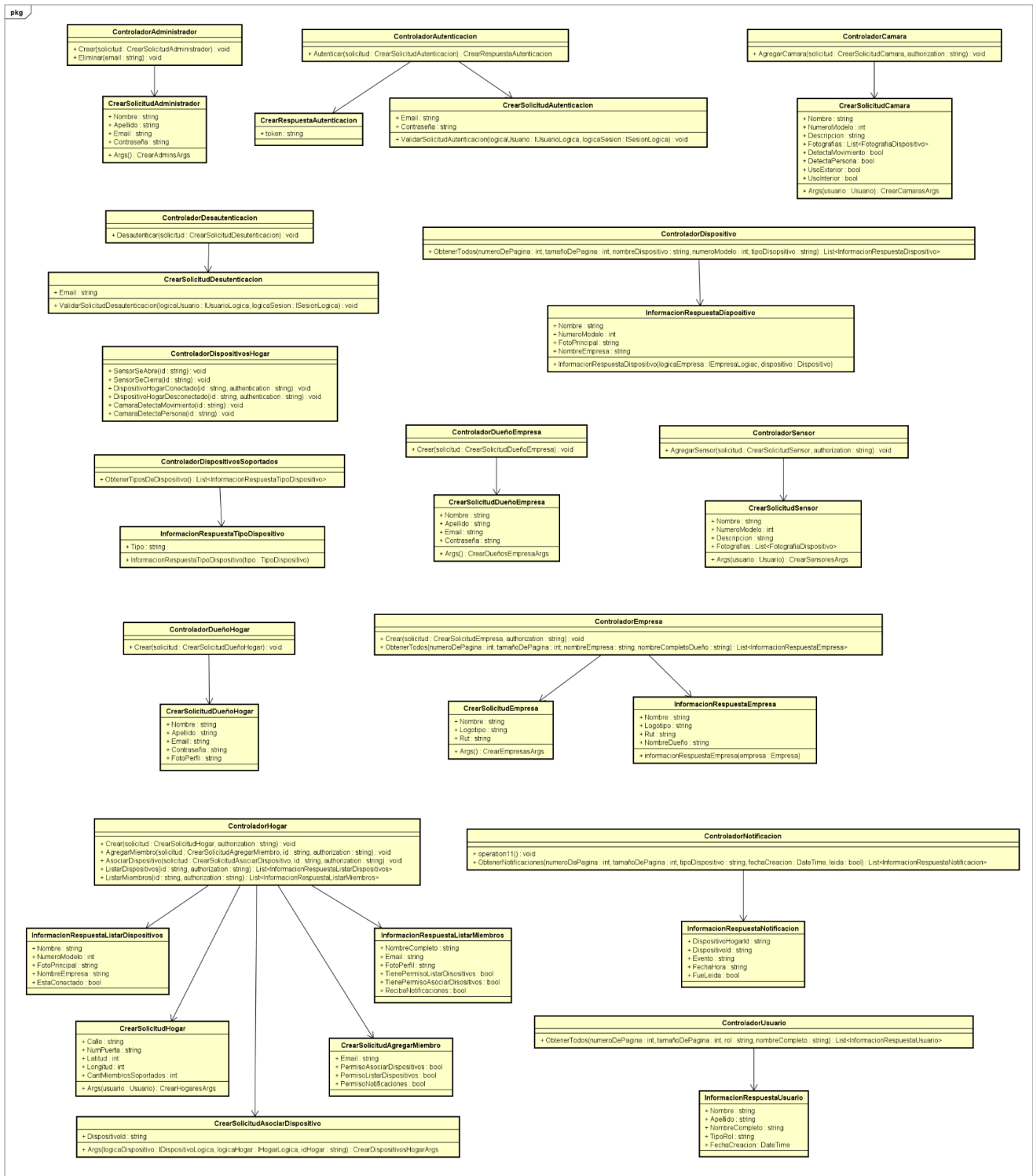


Manejamos una clase ContextoSql que contiene todos los sets con nuestras clases para representar las tablas correspondientes dentro de la base de datos.

Los repositorios contienen todos aquellos métodos necesarios para poder interactuar con la base de datos de manera efectiva.

Utilizamos una relación de composición, ya que si el ContextoSql se destruye o deja de existir, el repositorio no puede seguir funcionando porque depende totalmente de esa instancia para realizar sus operaciones. La composición refleja una dependencia mucho más fuerte que una asociación. En este caso, el repositorio no puede funcionar sin el contexto, lo cual es una característica clave de la composición.

Diagramas de WebApi

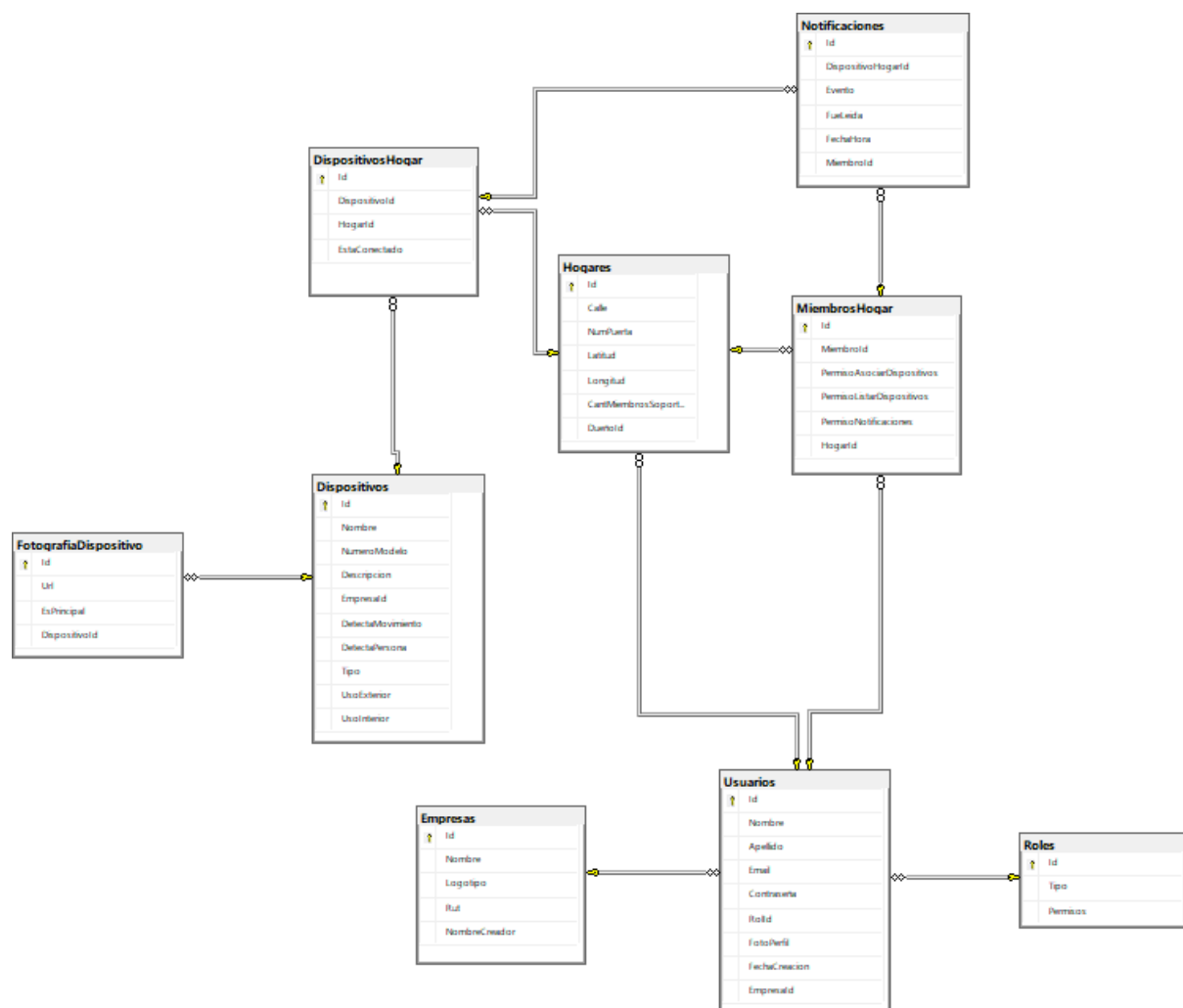


En el paquete están definidos todos los controladores y los modelos de entrada y salida. Los modelos nos sirven para no exponer nuestras clases mejorando la integridad. Además, al crear una nueva entidad solo debemos especificar los campos requeridos por el modelo, por lo que si cambia el dominio, el mismo no afectará a las aplicaciones que consuman nuestra API.

Dentro de WebApi también podemos encontrar los filtros de autenticación (comprueba que el usuario esté logueado), autorización (comprueba que el usuario tenga el permiso necesario para realizar una operación) y excepciones (mediante un diccionario que asigna comportamientos a excepciones específicas).

Decidimos incluir la lógica de la sesión dentro de este paquete ya que solamente es utilizada por los controladores, sus modelos y los filtros, que se encuentran dentro de WebApi. Al no tener otras responsabilidades fuera del contexto de los controladores y los filtros, nos pareció razonable que perteneciera aquí.

Modelo de Tablas

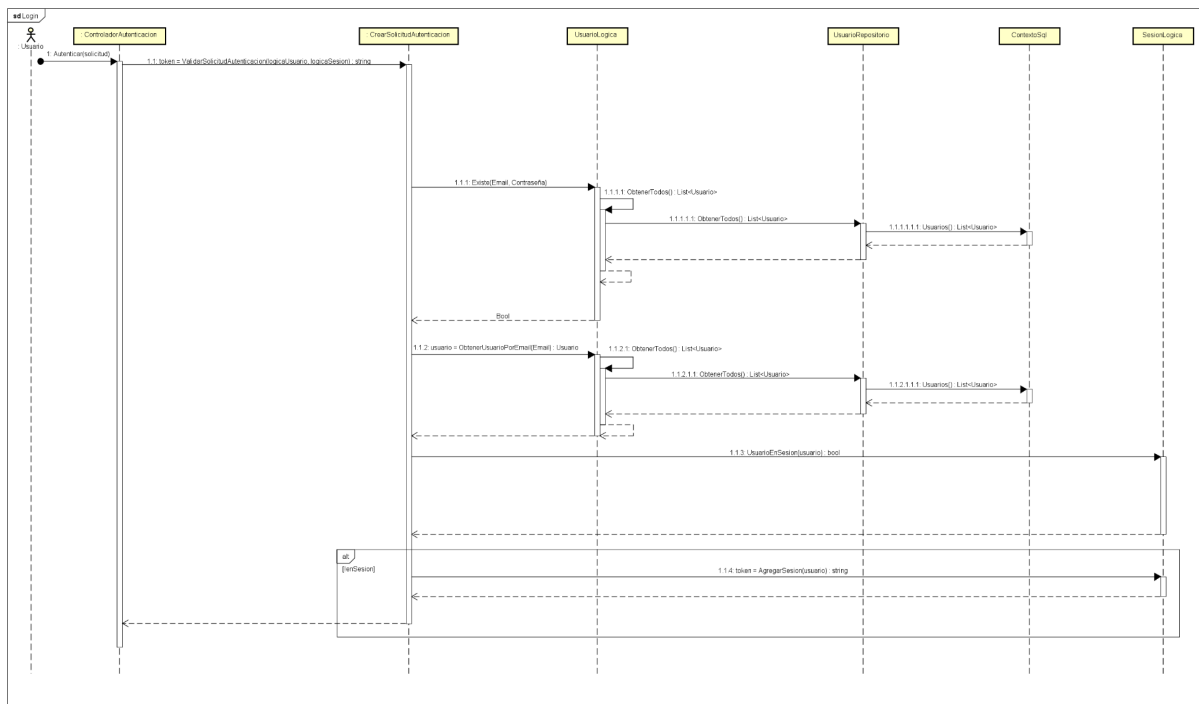


Este diagrama muestra el esquema actual de la base de datos generado por Entity Framework utilizando la metodología Code First. Las relaciones entre las tablas han sido correctamente definidas y reflejan la estructura lógica de nuestro sistema. Luego de realizar las migraciones correspondientes, esta es la versión actual del esquema.

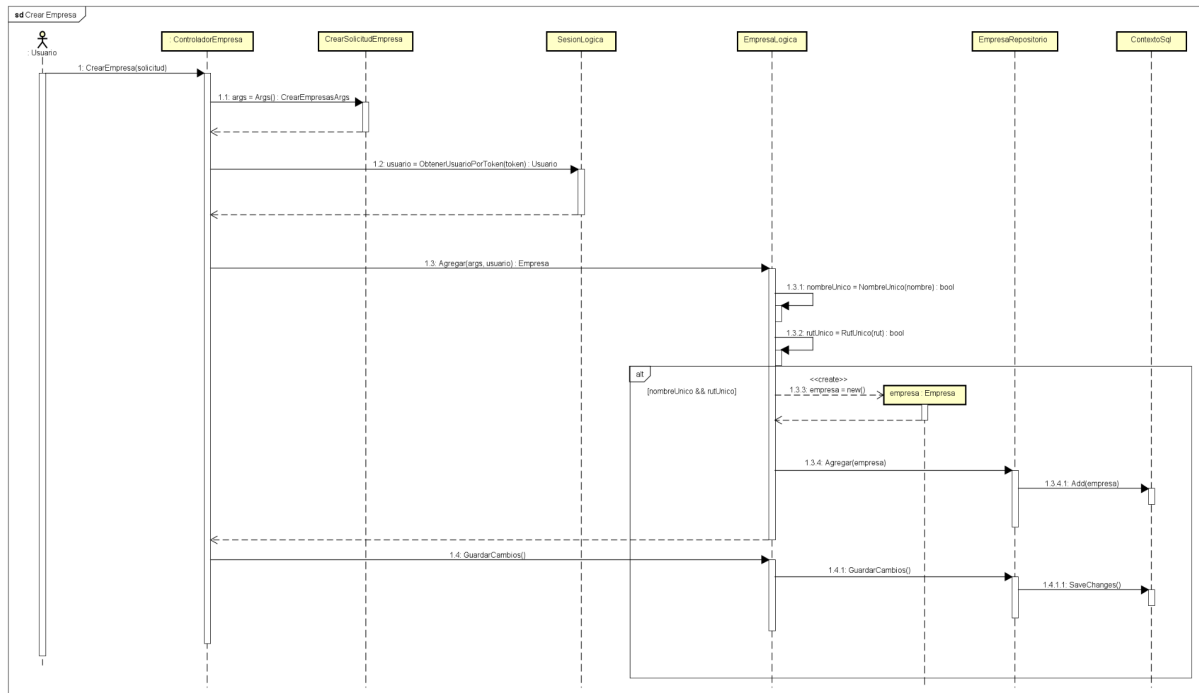
Diagramas de Interacción

En esta sección, veremos los diagramas de secuencia de algunas funcionalidades de nuestra aplicación, donde se puede ver cómo los distintos componentes del sistema funcionan en conjunto para llevar a cabo las principales funcionalidades del mismo. Nos centramos en seleccionar algunas de las funcionalidades que consideramos de crítica importancia para nuestro sistema, como lo pueden ser el login exitoso de un usuario, la creación de una empresa y añadir un miembro a un hogar.

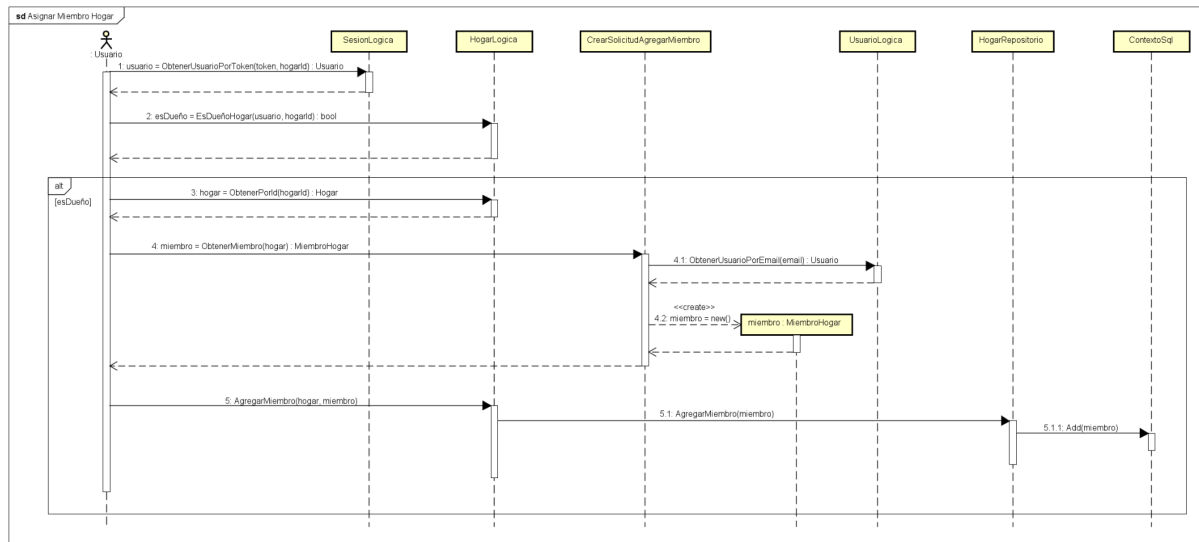
Login exitoso: Muestra los pasos necesarios para autenticar a un usuario en el sistema.



Creación de empresa exitosa: Detalla el proceso de creación de una nueva empresa, validando que no exista ninguna en el sistema con el mismo nombre o rut.



Añadir miembro a hogar exitoso: Representa el flujo necesario para añadir un nuevo miembro a un hogar existente.



Justificación del Diseño

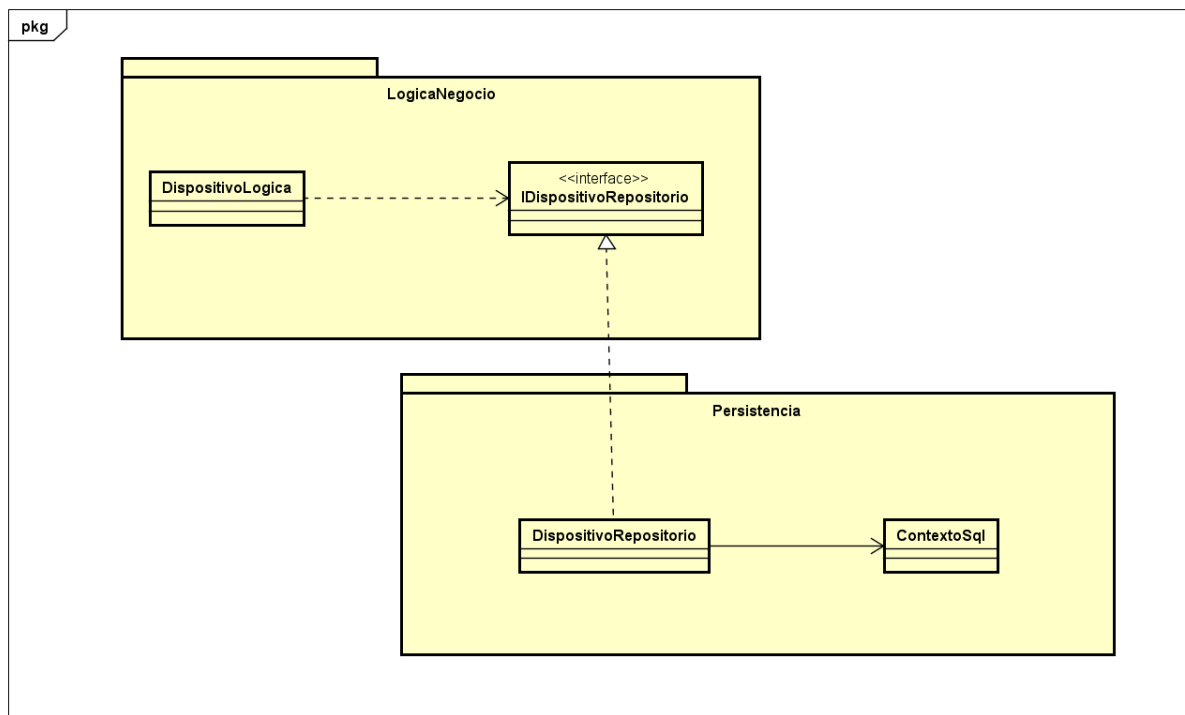
En el diseño de nuestro sistema utilizamos la inyección de dependencia para separar el sistema en capas, manteniendo una dependencia entre las mismas con respecto a paquetes de interfaces claramente definidas y no con respecto a las implementaciones. Esto nos permite cambiar implementaciones concretas sin modificar el código existente, por ejemplo, si necesitamos cambiar la lógica de acceso a datos en el futuro, solo basta con modificar la implementación inyectada en lugar de refactorizar múltiples clases, mejorando la mantenibilidad, la modularidad y la escalabilidad del sistema.

Una decisión clave en nuestro diseño es la utilización del ciclo de vida Scoped para las instancias de las lógicas y repositorios en la aplicación. Al utilizar Scoped, aseguramos que las mismas instancias sean reutilizadas dentro de un mismo contexto, lo que mejora el rendimiento y evita la recreación innecesaria de objetos, como ocurriría si usáramos Transient, que genera una nueva instancia en cada uso. Esto es especialmente útil cuando múltiples componentes necesitan acceder a las mismas instancias de lógica dentro de una operación, asegurando consistencia en los datos y evitando sobrecargas de memoria.

Para el acceso a datos, utilizamos Entity Framework con SQL Server. Tenemos un Contexto SQL que actúa como intermediario entre la aplicación y la base de datos. La arquitectura de acceso a datos sigue una separación de responsabilidades entre la lógica de negocio y la capa de persistencia, lo que mejora la mantenibilidad del código. Cada entidad tiene su correspondiente Repositorio, el cual es responsable de las interacciones directas con la base de datos.

En cuanto al manejo de excepciones, hemos implementado un filtro de excepciones personalizado (ExcepcionFiltro), que se añade a nivel global en la configuración del controlador. Este filtro captura y gestiona las excepciones no controladas, permitiendo una respuesta uniforme y clara ante errores inesperados. Al configurar este filtro en un solo lugar, mejoramos la mantenibilidad y garantizamos que los errores se manejen de manera consistente en toda la aplicación.

Patrón de Diseño Adapter



El patrón Adapter permite que dos interfaces incompatibles trabajen juntas al traducir las operaciones de una clase en términos que otra pueda entender. En nuestro sistema, utilizamos este patrón para resolver el problema de desacoplar la lógica de negocio de los detalles específicos de acceso a la base de datos. Este desacoplamiento se logra mediante el uso de un repositorio, en este caso `DispositivoRepositorio`, que actúa como intermediario entre la lógica de negocio (`DispositivoLogica`) y la capa de persistencia (representada por `ContextoSql`).

En este caso, utilizamos el patrón Adapter para traducir las operaciones que espera la lógica de negocio (`DispositivoLogica`) en términos que el sistema de persistencia (`ContextoSql`) pueda entender.

La lógica de negocio trabaja con la interfaz `IDispositivoRepositorio`. Dicha interfaz define las operaciones que la lógica espera, y abstrae los detalles de cómo se accede a la base de datos.

La clase `DispositivoRepositorio` actúa como el Adapter. Implementa la interfaz `IDispositivoRepositorio`, lo que le permite a la lógica de negocio interactuar con el repositorio sin conocer los detalles internos de cómo se realizan las operaciones de persistencia. Esta clase traduce las llamadas de `IDispositivoRepositorio` a operaciones compatibles con el contexto `ContextoSql`.

El Adapter (en este caso `DispositivoRepositorio`) traduce las solicitudes de la lógica de negocio en acciones que el contexto de persistencia (`ContextoSql`) puede ejecutar, como buscar un dispositivo en el `DbSet<Dispositivo>`.

ContextoSql actúa como el adaptado. Este contexto no implementa directamente la interfaz IDispositivoRepositorio, ya que su objetivo es interactuar con la base de datos y no exponer una interfaz específica para la lógica de negocio. Por lo tanto, requiere de una clase adaptadora para que la lógica de negocio no dependa directamente de los detalles de la persistencia.

De esta manera, también utilizamos este patrón para el resto de nuestras lógicas (DispositivoHogar, Empresa, Hogar, Notificacion y Usuario) y sus respectivos repositorios.

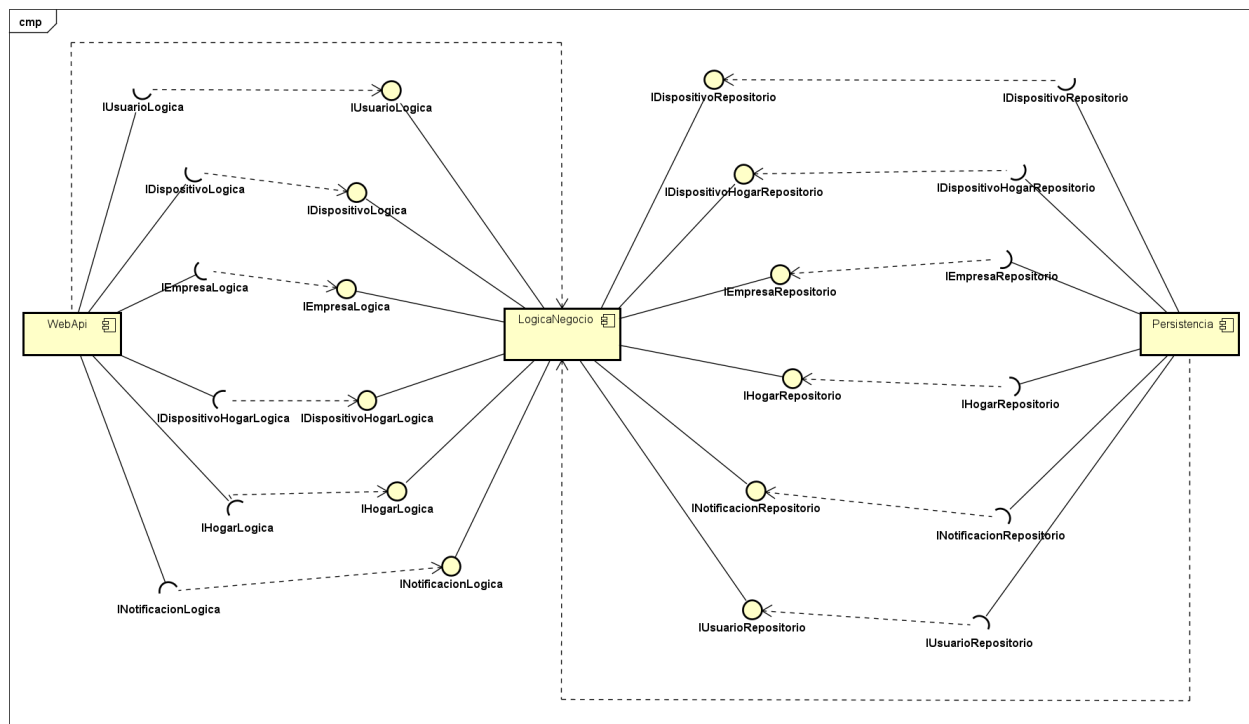
Beneficios en términos de SOLID:

- Principio de Responsabilidad Única (SRP): La lógica de negocio, el repositorio y el contexto de base de datos tienen responsabilidades claramente definidas y separadas.
- Principio Abierto/Cerrado (OCP): Podemos añadir nuevos repositorios sin modificar la lógica de negocio existente, lo que hace que el sistema sea fácilmente extensible.
- Principio de Inversión de Dependencias (DIP): La lógica de negocio no depende directamente de los detalles de la persistencia, sino de una abstracción, lo que garantiza un bajo acoplamiento y facilita los cambios en la persistencia sin afectar al negocio.

También se siguen ciertos patrones GRASP:

- Experto: A través del uso de los repositorios, nos aseguramos que las clases que tienen la información suficiente para realizar una tarea son las que la ejecutan. El DispositivoRepositorio tiene el conocimiento sobre cómo interactuar con la base de datos, mientras que la lógica de negocio (DispositivoLogica) se mantiene enfocada en las reglas de negocio sin preocuparse por los detalles de persistencia.
- Bajo Acoplamiento: Utilizando el patrón Adapter, la lógica de negocio queda desacoplada de los detalles de persistencia. Este bajo acoplamiento facilita el mantenimiento y la evolución del sistema.
- Alta Cohesion: Cada clase tiene una responsabilidad claramente definida. DispositivoLogica se encarga de la lógica de negocio, mientras que DispositivoRepositorio se encarga de la interacción con la base de datos. Esto refuerza la cohesión en cada componente y facilita la mantenibilidad.

Diagrama de Componentes



Decidimos implementar estos componentes porque el sistema fue diseñado para operar a través de capas interconectadas, alineadas con el principio de separación de responsabilidades. Este enfoque, combinado con el uso del patrón de inyección de dependencias, nos ofrece múltiples ventajas:

- Independencia entre capas: Cada capa, desde la Web API, pasando por la Lógica de Negocio, hasta la Persistencia, funciona de manera autónoma y está claramente definida.
- Mantenibilidad y comprensión: Al cumplir con el principio de responsabilidad única, donde cada componente se centra en una función específica, el sistema se vuelve más fácil de entender. Por ejemplo, las reglas de negocio están aisladas en la capa de Lógica de Negocio, mientras que la gestión de datos se maneja en la capa de Persistencia. Esto no solo facilita la adición de nuevas funcionalidades, sino que también permite corregir errores rápidamente.
- Facilidad para gestionar cambios: Una de las grandes ventajas de esta arquitectura por componentes es que cualquier cambio o actualización se puede realizar sin necesidad de modificar todo el sistema. Por ejemplo, si se requiere modificar la lógica de Usuario, solo se actualizará el componente correspondiente en la Lógica de Negocio y su Repositorio en la capa de Persistencia en caso de ser necesario. Esto reduce significativamente el tiempo requerido para implementar hotfixes o nuevas características.